Compute-Efficient DUST: Adaptive Population Allocation and Structured Perturbation Multiplexing

Team 27: Nikas Lukyanov (nl2951), David Fu (dyf2101), Junyang Ma (jm5914), Patrick Shen (pts2125), Srabon Nath (skn2134)

Goals. DUST [1] trains transformers without backpropagation by perturbing activations and estimating output errors from reward-weighted noise, in the spirit of evolution strategies [3]. Larger perturbation populations improve gradient quality but increase compute, making population size a practical bottleneck. We study two approaches:

1. **Adaptive population allocation.** Dynamically assign more perturbation draws to layers or module groups with unstable gradient estimates and fewer to groups whose estimates are reliable. We test whether better allocation can match baseline performance with a smaller total population.
2. **Structured perturbation multiplexing.** Perturb multiple module groups in the same suffix-forward pass (the forward pass from the perturbed layer to the loss) using structured signed codes, then estimate their individual contributions from the shared loss signal. We test whether this can reduce the number of suffix-forward evaluations required for each update.

The central question is whether DUST can be trained with fewer perturbation samples and suffix-forward evaluations without sacrificing gradient quality or final model performance. We also investigate how these tradeoffs change as model parameter count increases while the perturbation budget and number of training tokens remain fixed.

Challenges. Adaptive allocation must estimate gradient uncertainty cheaply enough that measurement overhead does not eliminate its savings, while overly aggressive reductions may degrade gradient quality or starve difficult layers. Multiplexing introduces a different problem: simultaneous perturbations affect the same downstream loss, so structured codes may not perfectly separate their effects in a nonlinear transformer. Increasing model size may change which groups need more samples and how strongly simultaneous perturbations interfere. A fixed population also does not imply fixed compute: larger models and perturbations at earlier layers can require more work per draw.

Approach. We build on the official DUST implementation [2]. For adaptive population allocation, we periodically split each module group's perturbation population into independent subsets and compare their gradient estimates. High agreement suggests that additional draws provide diminishing returns; low agreement suggests that a larger population may be useful. We redistribute a fixed global perturbation budget toward uncertain groups while maintaining a minimum allocation for every group. We compare DUST's fixed allocation with gradient-agreement and variance-based policies, and vary how frequently allocation is recomputed. Separate runs at smaller global budgets test whether reallocation actually reduces the population needed to match the baseline.

For structured perturbation multiplexing, we perturb several module groups in one suffix-forward evaluation using Hadamard-style signed codes, which are exactly orthogonal across a code sequence. We correlate observed loss changes with the codes to estimate each group's contribution. We begin with two multiplexed groups and increase the multiplexing degree until gradient quality deteriorates. Because the transformer is nonlinear, exact separation is not assumed; we measure estimator variance, empirical bias relative to backpropagation gradients, and cosine similarity. We compare structured codes against i.i.d. random Rademacher sign codes (only

approximately orthogonal) at matched group-draw counts and noise scales. All evaluations needed to apply a code sequence are included in the compute accounting.

We compare four variants: baseline DUST (A), adaptive population allocation (B), structured perturbation multiplexing (C), and both methods combined (D). This design tests the benefit of reallocating draws, sharing suffix-forward evaluations, and combining the two approaches.

Performance Evaluation. We first evaluate A–D on identical checkpoints and batches within each model size. We measure cosine similarity with exact backpropagation gradients, estimator variance and empirical bias over repeated noise seeds, perturbation count, suffix-forward count, and estimated FLOPs. These diagnostics identify promising configurations before full training. For adaptive allocation, we track the distribution of draws across layers and the cost of recomputing allocations. For multiplexing, we measure gradient quality and runtime as more groups share each forward evaluation.

Promising configurations are trained on matched data and token budgets, using the same optimizer family and comparable hyperparameter-tuning budgets. We report held-out mean cross-entropy per token, held-out loss versus FLOPs and wall-clock time, GPU-hours, and training-step latency. At each model size, population sweeps test whether B–D can match A's held-out loss with fewer draws or less compute.

For model-size scaling, we hold the global perturbation budget per update and training-token budget fixed while increasing parameter count, repeating the sweep at several population budgets. All variants receive the same total budget; adaptive methods change only its distribution across module groups, and for multiplexing each perturbed group in a shared evaluation counts separately. At each budget, we plot held-out loss, gradient alignment, FLOPs, and GPU-hours against parameter count, with a backpropagation reference at each size to separate gains from model capacity from changes in the effectiveness of perturbation-based training. This tests whether our methods preserve or improve DUST's performance as models grow, and whether any loss improvement from larger models justifies their additional compute. Hardware is held constant across methods within each size; any changes across sizes are documented.

Implementation Details. We use Columbia's Insomnia cluster through Slurm and modify the official PyTorch DUST implementation [2]. PyTorch Distributed and NCCL support runs requiring multiple GPUs, while profiling measures runtime, memory use, and communication overhead. We begin with the released 37.7M-parameter transformer and FineWeb setup, then implement and validate the smaller configurations used in the paper's model-size study. Short runs and small populations support estimator and hyperparameter sweeps before longer experiments. We target a common 10M-token budget for the model-size study, matching DUST's experiment; if pilot measurements require a smaller budget, we use it consistently across all compared sizes and methods. Key training comparisons are repeated across multiple random seeds. Backpropagation is used for gradient diagnostics and separate reference runs, never to update the proposed DUST variants.

Planned Demo. We visualize how adaptive DUST reallocates perturbation draws across the model and how multiplexing changes gradient quality as more groups share each forward pass. Final plots compare A–D on gradient quality versus FLOPs, held-out loss versus wall-clock time and GPU-hours, and held-out loss versus parameter count at fixed perturbation and token budgets. Together, these plots show whether the proposed savings remain useful as model size increases.

References

[1] S. Dahal, B. Mandal, S. Gülbahar, and A. Vegesna. *Dust: Pretraining Transformers Without Backpropagation*. Q Labs, 2026. See Section 4.1 and Figure 4 for model-size experiments, and Appendices A and E for architecture and population accounting.

[2] Q Labs. *DUST Official Implementation*. GitHub, 2026.

[3] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever. *Evolution Strategies as a Scalable Alternative to Reinforcement Learning*. arXiv:1703.03864, 2017.